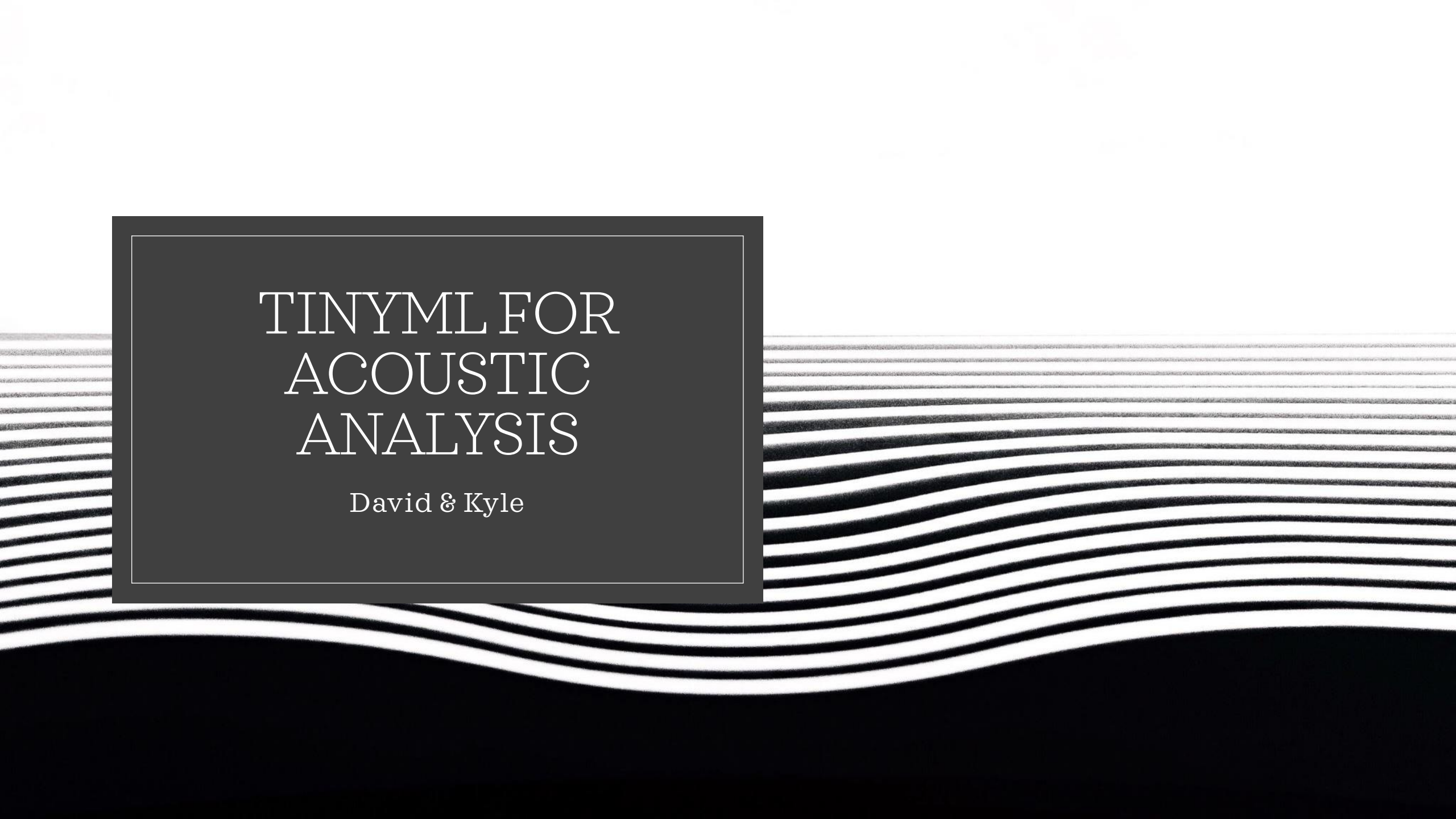


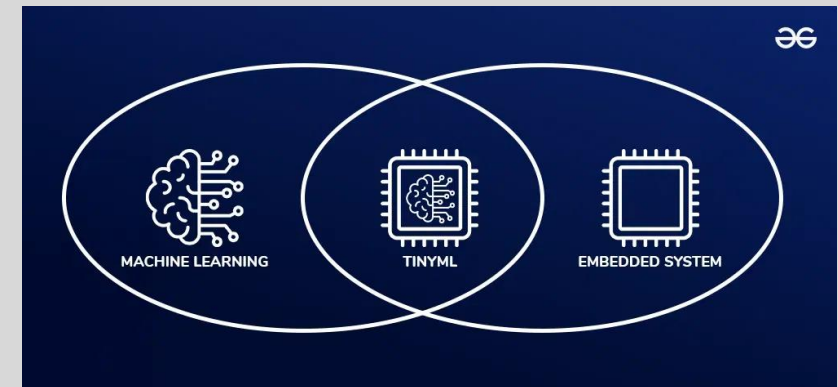


TINYML FOR ACOUSTIC ANALYSIS

David & Kyle

What is TinyML?

- Practice of deploying machine learning models on low-power embedded devices
- Instead of sending data to the cloud:
 - Data is collected locally
 - AI runs directly on the device
 - Decisions are made in real time
- Intelligence without requiring internet connectivity



Real World Applications and Growth

- Applications:

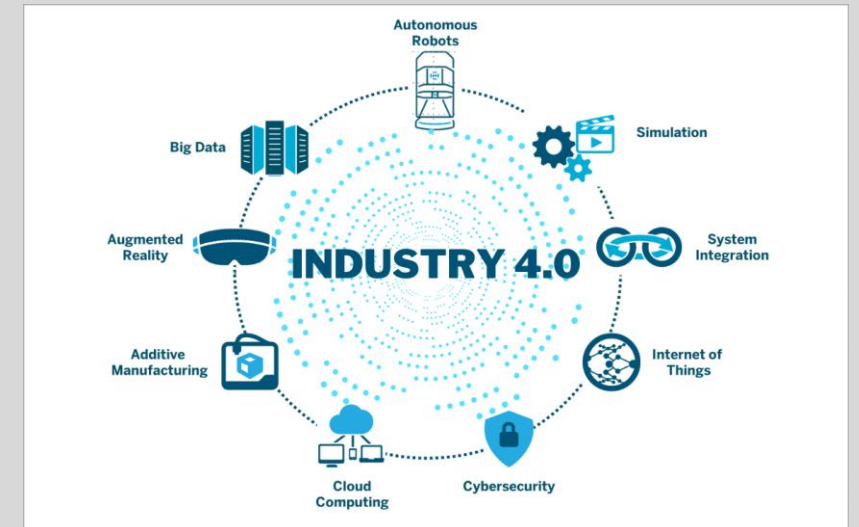
- Smart watches
- Smart speakers
- Autonomous drones
- Manufacturing
- Environmental monitoring
- Healthcare devices



- Market growth and future trends:

- Industry 4.0
- Edge AI
- Smart sensors
- IoT expansion

- Billions of devices will eventually contain embedded AI



Why TinyML is Important

Traditional AI

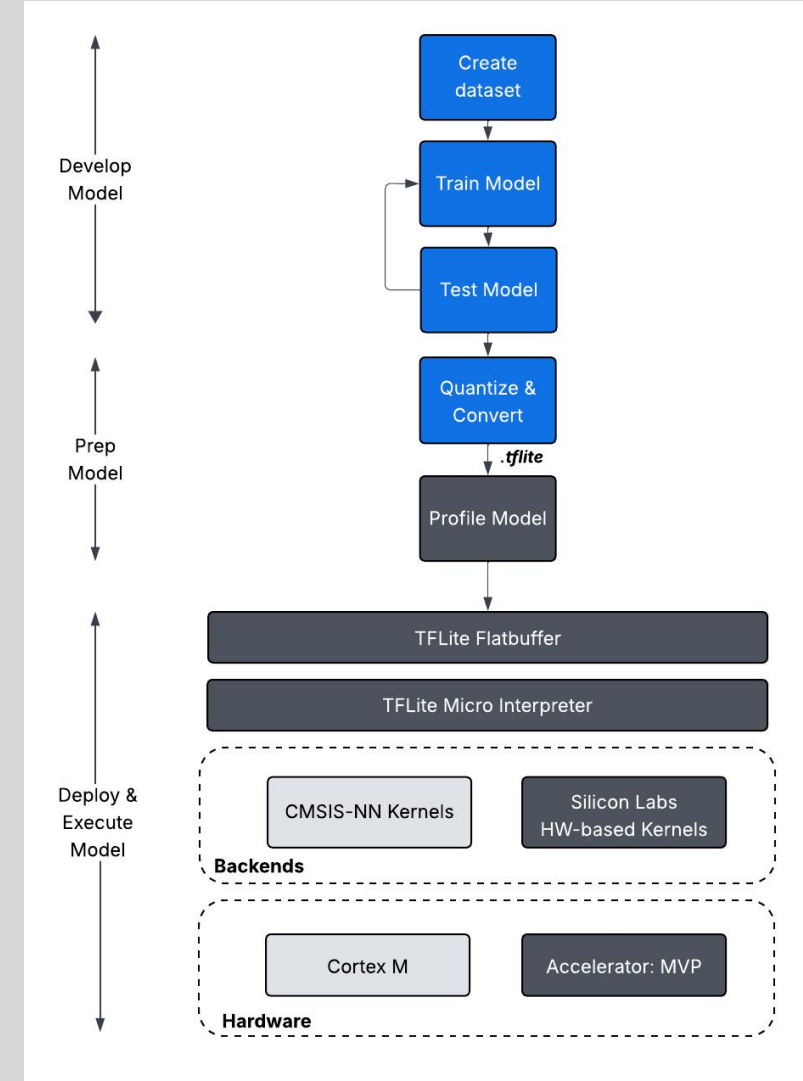
- Sensor -> Cloud -> AI processing -> Result
- Problems:
 - Internet required
 - Higher latency
 - Privacy concerns
 - Higher power consumption

TinyML

- Sensor -> Embedded Device -> Result
- Benefits:
 - Real-time decisions
 - Low power consumption
 - Improved privacy
 - Reduced communication requirements

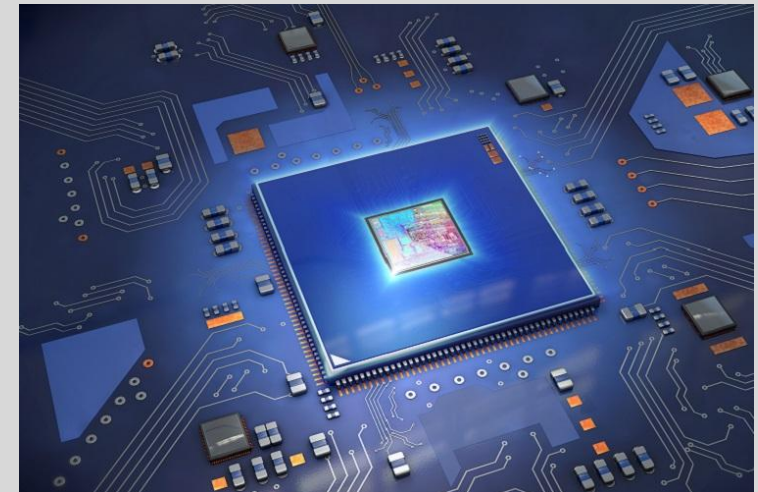
TinyML Architecture

- General Workflow:
 - Collect sensor data
 - Train model on computer
 - Optimize model
 - Deploy to microcontroller
 - Perform inference locally



Challenges of TinyML

- Microcontrollers have:
 - Limited RAM
 - Limited Flash
 - Limited CPU power
 - Battery constraints
- For example, a PC could have 32GB ram while a microcontroller has around 512KB
- Limited processing power:
 - Don't have powerful CPUs or GPUs
 - Complex AI models may take too long to run
 - Real-time applications require fast inference



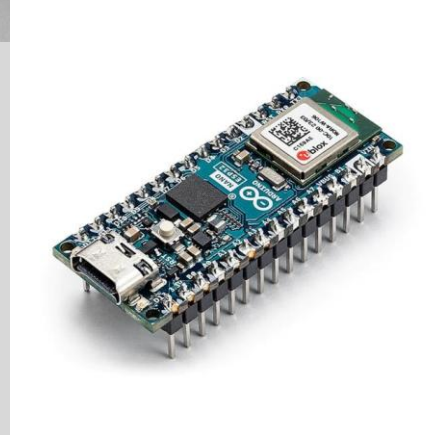
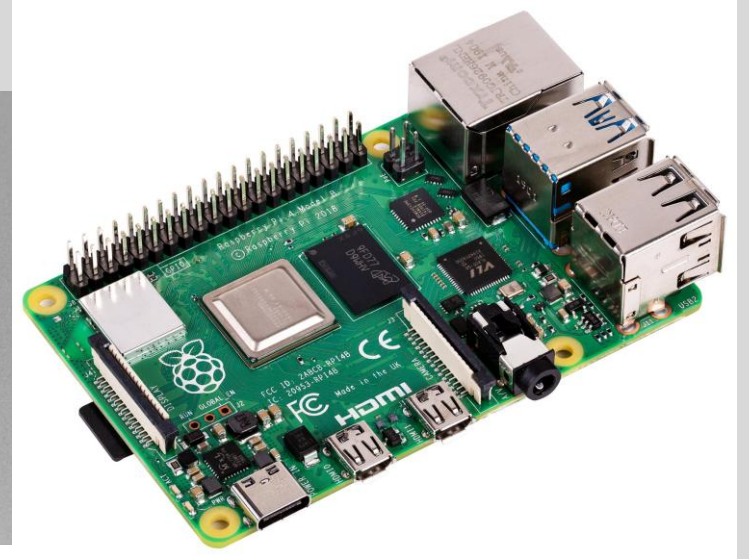
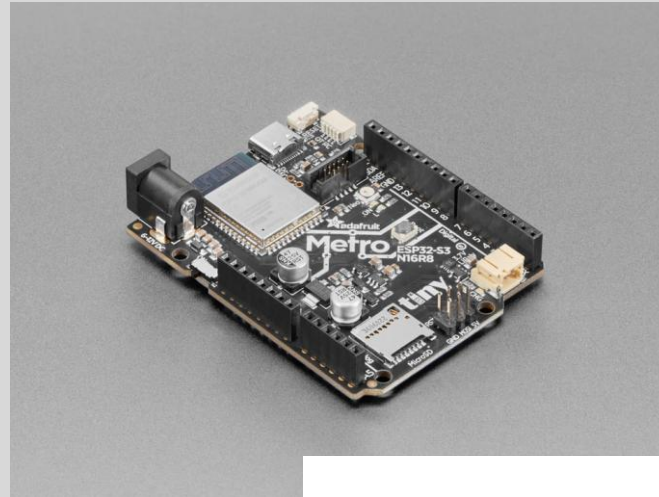
Hardware Used in TinyML

- **Common Platforms**

- ESP32-S3
- STM32
- Arduino Nano 33 BLE Sense
- Raspberry Pi Pico

- **Why ESP32-S3?**

- Our capstone already uses:
 - ESP32-S3
 - MEMS microphone
 - Wi-Fi communication
 - This makes it an ideal TinyML platform.

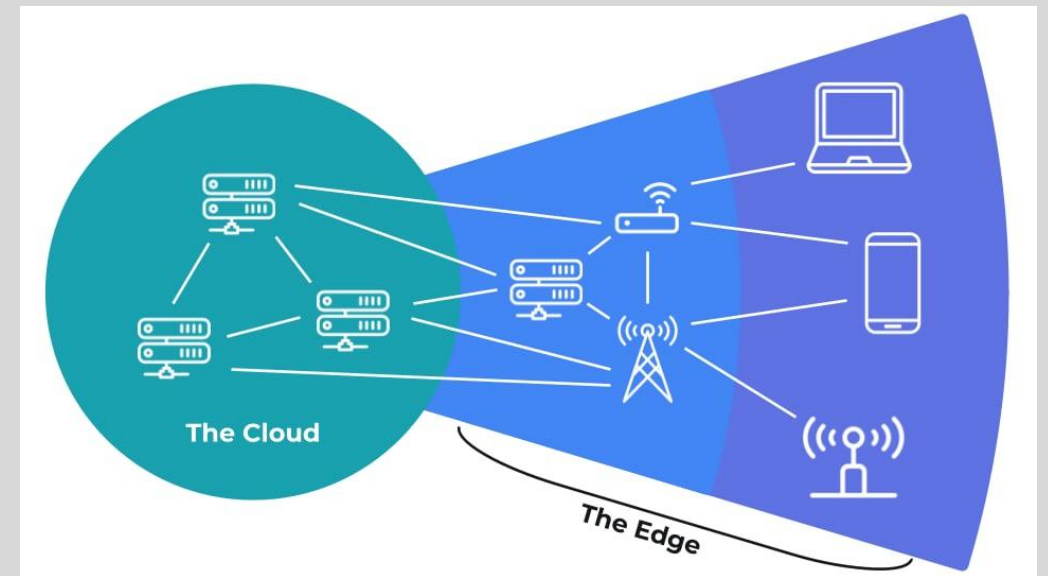


TinyML Development Tools

- **TensorFlow Lite for Microcontrollers**
 - Lightweight ML framework
 - Runs neural networks on microcontrollers
- **Edge Impulse**
 - Data collection
 - Model training
 - Model deployment
- **Advantages**
 - Easy deployment
 - Optimized for embedded systems
 - Supports ESP32 devices



TensorFlow Lite



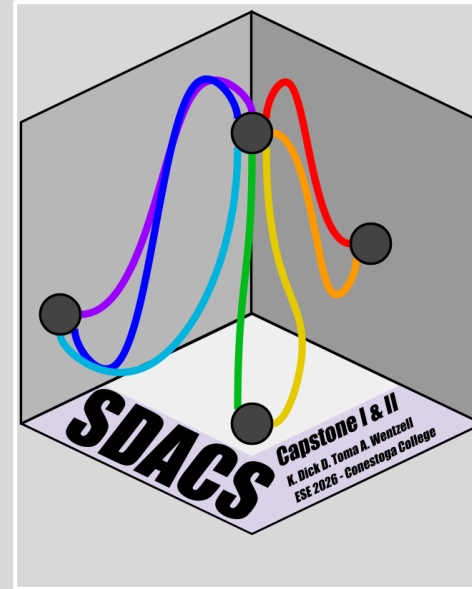
Integration to SDACS – Current Workflow

- **Current Functions**

- Sound Pressure Level (SPL)
- FFT Analysis
- Frequency Response
- Room Acoustic Measurements
- Wireless Data Collection

- **Current Workflow**

- Microphone → ESP32-S3 DSP → TinyML inference → MQTT/Node-RED → Flutter AI result display
- The system measures sound but does not currently interpret it.



Integration to SDACS – New Workflow

- Microphone → DSP Processing → TinyML Model → AI Prediction → App Display
- **AI Capabilities:**
- The system could automatically:
 - Classify room types
 - Detect echo issues
 - Detect excessive reverberation
 - Evaluate speech intelligibility
 - Recommend acoustic treatments
- The device becomes an intelligent acoustic assistant.



How it Benefits Our Capstone

- **Technical Benefits**

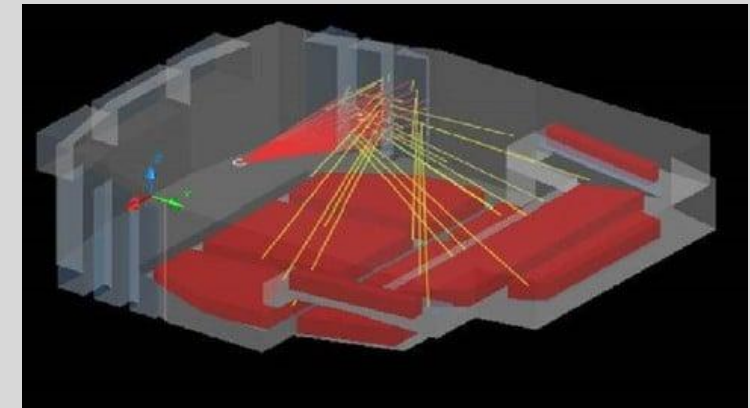
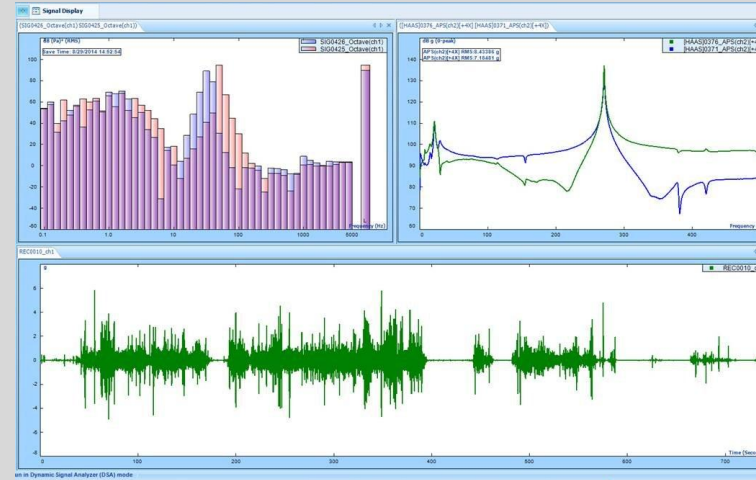
- Real-time acoustic intelligence
- Local processing
- Reduced server workload
- Faster analysis

- **Educational Benefits: Combines**

- Artificial Intelligence
- Machine Learning
- Embedded Systems
- Signal Processing
- IoT
- Acoustic Engineering

- **Industry Relevance**

- Growing use of Edge AI and TinyML in modern engineering systems.



TinyML Limitations & Better AI/ML Alternatives

TinyML limitations for SDACS

- Needs a labelled acoustic dataset before predictions are reliable
- ESP32-S3 RAM, flash, and CPU limit model size and inference speed
- Raw-audio ML adds DSP, memory, and battery load
- Harder to debug than the current SPL / FFT telemetry pipeline

More practical AI/ML path

1. Rule-based acoustic intelligence now
Use SPL, FFT bands, noise floor, and thresholds to flag noise, echo, and low-frequency buildup.
2. RPi5 / Node-RED or Flutter-side ML next
Keep ESP32 nodes focused on sensing; run classifiers and recommendations on the server/app.
3. TinyML later
Deploy a small feature-based model only after the dataset and benchmarks are proven.

Current SDACS
SPL + FFT Data

Server/App AI
Rules or ML

Future TinyML
On-node inference

Recommendation: present TinyML as future work; implement server/app-side intelligence first for a stronger, lower-risk SDACS demo.

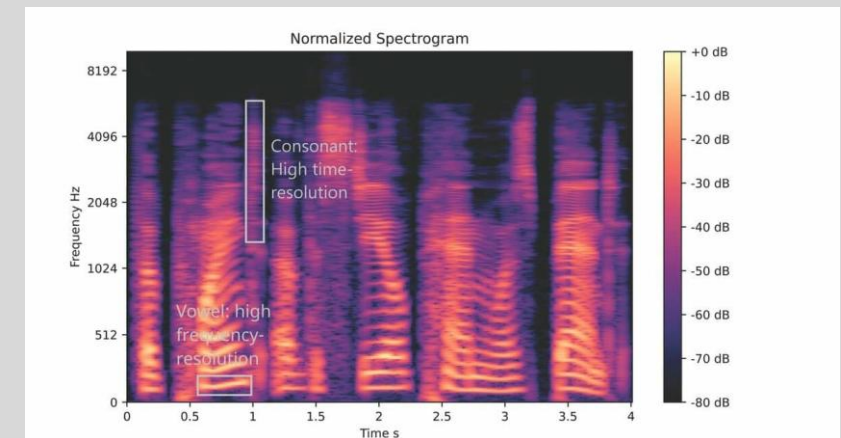
Conclusion

- **Key Takeaways**

- TinyML enables AI on low-power embedded devices.
- ESP32-S3 hardware supports TinyML applications.
- SDACS could use TinyML to automatically analyze room acoustics.
- AI would transform the project from a measurement system into an intelligent acoustic analysis platform.

- **Future Work**

- Collect acoustic datasets
- Train room-classification models
- Deploy TinyML models on ESP32-S3
- Integrate AI results into Flutter application



References

- [1] Edge Impulse, "Edge Impulse: Edge AI Development Platform," Available: <https://www.edgeimpulse.com/>
- [2] Google AI for Developers, "LiteRT for Microcontrollers," Available: <https://ai.google.dev/edge/litert/microcontrollers/overview>
- [3] TinyML Foundation, "TinyML Foundation," Available: <https://tinyml.org/>
- [4] Espressif Systems, "ESP32-S3 Series," Available: <https://www.espressif.com/en/products/socs/esp32-s3>